

From Algorithm to Program

TC1037 · Algorithmic Thinking · Lecture 02

Dr. Leopoldo Rodríguez Salazar

Friday 25 September 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Tuesday's algorithm, typed in

```
1 distance = input("Distance in km: ")
2 time      = input("Time in hours: ")
3
4 speed = distance / time
5
6 print("Average speed:", speed)
```

Does this work? **Which line?**

The machine is less forgiving than I was

```
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

- It tells you the **operator**: /
- It tells you the **types**: str and str
- It is not cryptic. It is a **diagnosis**.

Failure 1 — everything the user types is text

✗ Broken

```
distance = input("Distance: ")    # "100"  <- text
speed = distance / time            # TypeError
```

✓ Fixed

```
distance_km = float(input("Distance: "))
speed = distance_km / hours        # 50.0
```

`input()` returns `str`. Always. Even when every character is a digit.

The version that does not crash

```
a = input("First number: ")    # "2"  
b = input("Second number: ")  # "3"  
print(a + b)                  # 23    <- not 5
```

No error. No warning. **Wrong.**

Anatomy of a program

Part	What it does	Example
Comment	a note for humans	<code># calculate the speed</code>
Variable	a named box holding a value	<code>distance = 100</code>
Assignment	puts a value in the box	<code>speed = d / h</code>
Expression	something with a value	<code>d / h</code>
Input	gets a value (as text)	<code>input("Distance: ")</code>
Output	shows a value	<code>print(speed)</code>
Function call	uses code someone else wrote	<code>float(...), round(...)</code>

READ $\rightarrow$ `input()` `<-` $\rightarrow$ = DISPLAY $\rightarrow$ `print()`

Failure 2 — Python has its own opinion

1	()	parentheses
2	**	exponent — right to left
3	-x	negation
4	* / // %	left to right
5	+ -	left to right
6	< <= > >= == !=	comparison
7	not and or	logical

When you are unsure, parentheses cost nothing and buy certainty.

The one that will cost somebody marks

```
g1, g2, g3 = 90, 80, 70

average = g1 + g2 + g3 / 3          # 193.33
average = (g1 + g2 + g3) / 3       # 80.0
```

It runs. It prints a number. **The number is nonsense.**

Running is not a test.

- The average program ran. It was wrong.
- The `a + b` program ran. It was wrong.

Your only defence today: **know what you expect before you press run.**

Built-in functions — funciones predefinidas

```
abs(-7)           round(3.14159, 2)   max(3, 9, 2)
len("algoritmo")  int("42")                           float("2.5")

import math
math.sqrt(16)      math.pi              math.floor(3.9)
```

The distinction that matters

`round(3.7)` has a **value**: 4. You can store it.

`print(3.7)` has no value. It has an **effect**.

Exercise 1 — Predict, then run

Individual · 10 minutes

Open `examples/03_precedence.py`. **Do not run it.**

Write your prediction for all nine lines on paper. *Then* run it and mark each right or wrong.

Hand in the paper — **including the wrong ones**.
The wrong ones are the useful part. They are not penalised.

Exercise 2 — Trip cost

Plenary · built together · 20 minutes

Given distance (km), speed (km/h), fuel economy (km/litre) and price per litre:
display the travel time, the litres required, and the total cost in pesos.

1. **Algorithm on paper first.** I will look at it before anyone opens Thonny.
2. Code it from *your* algorithm.
3. Check: 200 km, 100 km/h, 12 km/l, \$24.50 $\rightarrow$ 2.0 h, 16.67 l, \$408.33

Exercise 3 — Celsius to Fahrenheit

Individual · 10 minutes

$$^{\circ}F = ^{\circ}C \times 9/5 + 32$$

Two required checks: $0 \rightarrow 32.0$, $100 \rightarrow 212.0$

If you wrote `c * 9 / 5 + 32` without thinking about precedence — were you **lucky** or **right**? Be able to say which.

1. **Homework 1** — BMI and seconds-to-time. Due **Sunday 27 Sep, 23:59**.
Handwritten algorithm is part of the submission.
2. Respondus practice exam — Monday 28 Sep, 23:59.
3. Levelling module: units 3, 4, 5.
4. Autoestudio: relational and logical expressions, decision structures.

Quick exam 1: Tuesday 29 September, first 20 minutes.

Algorithms · operator precedence · built-in functions · calculations

**A program that runs
is not a program that is correct.**